

Projet « récapitulatif » des notions cruciales qui devraient être acquises et maîtrisées (*préalables*)

- Ne pas renforcer une faiblesse dès qu'elle est identifiée aura comme effet secondaire de vous créer d'autres faiblesses sur des éléments rattachés directement ou indirectement à cette faiblesse;
- Capitaliser sur une force pour permettra d'assoir plus facilement de nouvelles connaissances et compétences sur des éléments futurs rattachés à cette force.

Objectifs

L'objectif « caché » de cet exercice est principalement de :

- Permettre de valider les éléments sur lesquels vous devez capitaliser et également travailler
- Identifier vos « faiblesses »;
- Identifier vos « forces »;
- Pratiquer la vulgarisation de théories complexes entre collègues;
- Rechercher l'information pertinente pour compléter les concepts moins bien maîtrisés;
- Mettre à votre disposition des sources afin de vous aider tout au long de la session;
- Amener les étudiants à appliquer les bons concepts de la POO dans une situation donnée.

Travail à effectuer en classe*

- Durée : 5 à 7* heures

Directives générales

- Ce travail doit être réalisé **seul**
- L'utilisation des IA génératives est interdite 
- La recherche sur les moteurs de recherche et autres ressources est autorisée 

Consignes / Contraintes

Vous devrez vous assurer de bien comprendre tous les concepts suivants :

- Concept de Programmation Orientée Objets (P.O.O.)
 - Utilisation adéquate des différentes visibilitées de membres, méthodes, etc...
 - 1 responsabilité → 1 classe
 - Classe != Conteneur de valeur
 - Héritage et polymorphisme
- Notions, méthodes et pratiques de débogage (*avancées*)
- Bonne pratique de programmation
 - Code propre et indenté
 - Commentaire là où pertinent
 - Modularité du code et réutilisation
 - Petites fonctions, petites méthodes, ...
 - Utilisation appropriée des structures de données
 - Différents types de paramètre
 - Gestion d'exceptions et validations
 - Pas de : GOTO, Break, continue, variable globale → **sans justification**
- Utilisation d'un dépôt de source / Gestionnaire de version / Gestionnaire de code
 - Gestion des branches
 - Gestion des conflits
 - Utilisation adéquate et non pas seulement pour déposer les travaux à la fin...
- Les notions de bases de données
 - Requêtes et manipulations de données
 - Modélisation et normalisation
- Concept de recherche efficace sur internet pour identifier de nouveaux opérateurs, nouvelles méthodes ou fonctions, etc...
- Projet de tests et tests
- Conception de pseudocode (*algorithmique*) avant de coder
 - Se faire une tête précise sur la logique de traitement/résolution avant d'ouvrir VS
- Analyse, production de documents d'analyse et modélisation
 - Format d'un document professionnel : Page titre, pagination, table des matières, etc...
- Prototype et développement agile (*itératif*)
- Notions de plagiat et tricherie
 - Utilisation de l'IAg → indication de la provenance des codes non écrit par vous
- Gestion du temps et du travail
 - Création d'échéancier, division en tâche et sous-tâche, estimation de temps et validation des estimations, etc...
- Post-mortem

IMPORTANT : Il ne devrait **JAMAIS** y avoir **aucun bout de code** que vous ne soyez pas en mesure d'expliquer **EXACTEMENT** ce qu'il fait dans votre code/programme (**Attention aux copier-coller peu importe la provenance**)

Situation concrète à réaliser et implémenter

Pour la situation proposée, réfléchir et analyser les concepts de programmation orientée objet qui devraient être impliquées (*héritage, polymorphisme, override, overload, classe abstraite, interface, classe statique, etc.*) afin de vous permettre de « résoudre les différents sous problèmes » de la mise en situation.

Mise en situation

Vous devez implémenter une petite application native afin de permettre la recherche de livres pour une bibliothèque numérique où l'on doit gérer :

- Un grand nombre de livres avec leurs métadonnées (titre, auteur, date de publication, genre, ISBN, etc.)
- Des membres inscrits avec leurs coordonnées et leur historique d'emprunts
- Les emprunts et retours de livres, consultation des stocks disponibles en « temps réel »
- La possibilité de rechercher rapidement les livres selon différents critères

- ⇒ Vous devez modéliser et normaliser la base de données pour ce projet
- ⇒ Vous devrez prévoir quelques données pour chacune des tables afin de permettre de tester le bon fonctionnement de votre « programme »
- ⇒ Pour le moment vous ne développez que le programme qui permet de consulter l'information (*pas de connexion, aucune saisie, aucun ajout, aucun emprunt/retour*)
- ⇒ Il est fortement suggéré de concevoir votre programme avec le patron de conception MVVM
(*vous forcera à le pratiquer et devrait être plus rapide pour la suite des projets*)
- ⇒ Lorsque votre modélisation sera « réalisée » en discuter avec l'enseignant avant de l'implémenter afin d'éviter d'éventuelles problématiques lors de l'implémentation de

frais ?

Livrable de ce laboratoire (À remettre avant de quitter sur LÉA)

Dans un fichier archive .ZIP :

- Le script de création de votre base de données
- Le script d'insertion de données dans votre base de données
- La solution Visual Studio Community 2026 **complète** et **NETTOYÉE**
- **(NON APPLICABLE POUR CE TRAVAIL, MAIS LES PROCHAINS OUI)**
 - ***Un fichier texte ou Word, contenant tous vos prompts à des IAg***

Pas officiellement des livrables pour ce travail mais essentiel pour un travail ultérieur. Assurez-vous donc de l'avoir en note quelque part dans un fichier sur votre P:\ ou sur OneDrive

- Identifiez 3 forces, à partir des éléments énumérés dans la section « Consignes / Contraintes », qui font de vous un atout au sein d'une équipe de projet;
- Identifiez 3 faiblesses, à partir des éléments énumérés dans la section « Consignes / Contraintes », qui font que vous auriez besoin d'aide dans une équipe de projet.

Savoir identifier ses forces et ses faiblesses est essentiel pour un technicien informatique. Cela vous permet de capitaliser sur vos forces et améliorer vos faiblesses.